

COM106: Introduction to Databases

Introduction to SQL

COM106 Weekly Schedule - Lecture & Lab Delivery

Week 9 Lab is the in-class exam (Set Exercise, 90-minute class test, 40% of module mark).

Week	Lecture	Lab / Seminar
1	Module Introduction & DB Basics and Security	Lab 1: Getting Started with MYSQL
2	Introduction to Entity-Relationship Modelling	Lab 2: Creating ER Diagrams with draw.io
3	ER Modelling 2 & Mapping to the Relational Model	Lab 3: ER Diagram Design from Scenarios
4	Normalisation	Lab 4: Normalisation Practice
5	**Introduction to SQL (DDL, DML, DCL)** - SQL overview & data types - CREATE TABLE & primary keys - Integrity constraints (DEFAULT, UNIQUE, CHECK) - Foreign keys & ALTER TABLE - INSERT, UPDATE & DELETE	**Lab 5: Building a Database with SQL (businessdb)** - Creating database & tables in SQL (MySQL data types) - Referential integrity via ALTER TABLE - ALTER TABLE variations - INSERT, UPDATE & DELETE records - Exporting / backing up businessdb (mysqldump) - First SELECT exercises (projemp)

https://github.com/KodZone/COM106-MYSQL



Recap

- **Normalisation** can be used as a validation of the relational schema produced as the result of an ER Modelling process.
- The aim is to satisfy the following requirements:
  - The minimal number of attributes necessary without contains **multiple values or repeating groups.**
  - Attributes with a close logical relationship are found in the same table (**functional dependency**)
  - **Minimal redundancy** – each attribute appears only once (except for foreign keys)

- In designing a database, the proceed include the following stages:
  1. create an ER Diagram
  2. use the relational mapping rules to create a linked relational schema
  3. check that each relation is in **3NF** and if it is not
     - either decompose to **3NF**
     - or leave in **2NF** for performance reasons if justified by update rate
- **3NF** is normally sufficient for most aspects of database design.

SQL overview & data types

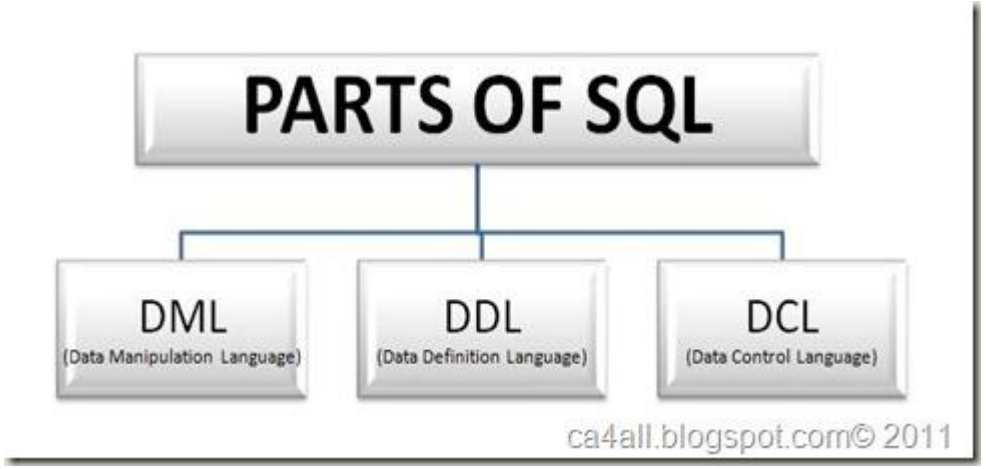
Structured Query Language (SQL) – An Introduction

What is SQL?

- SQL is a **Programming Language**, designed to manage data in a relational dB
  - Uses simple **declarative** statements
- It is a defacto **Industry Standard**.
- There is an ANSI standard - although there are many different versions and dialects.
  - For Instance, MS SQL Server 20XX uses a version know as Transact-SQL (or T-SQL), that have commands may vary slightly from ANSI-SQL or SQL used in other systems (e.g. MySQL)
  - Similar, MySQL Server have MySQL Procedural Language.

SQL Three Main Components

- **SQL consists of 3 main components:**



  - DDL - Data Definition Language
    - (used in the creation/destruction of relational tables, indexes, views etc.)
  - DML - Data Manipulation Language
    - (used for retrieval, appending, deleting, updating tables, etc)
  - DCL - Data Control Language
    - (used for creating table permissions, valid users, etc)

SQL Main Components - DDL

SQL consists of 3 main components:

DDL - Data Definition Language

(used in the creation/destruction of relational tables, indexes, views etc.)

e.g. **CREATE TABLE** EMP
(empnum smallint not null,
ename char(15),
salary smallmoney,
deptnum smallint**);**

As used in the Lab.

Indentation and layout are not required for execution
*They are **required for clarity**.*

CREATE TABLE EMP (empnum smallint not null, ename char(15), salary smallmoney, deptnum smallint);

SQL Main Components - DML

DML - Data Manipulation Language
(used for retrieval, appending, deleting, updating tables, etc)

e.g.

```
INSERT INTO EMP
VALUES (1, 'Smith', 10000, 12);
```

```
UPDATE EMP
SET salary = 20000
WHERE ename = 'Smith' ;
```

```
DELETE FROM EMP
WHERE ename = 'Jones';
```

```
SELECT ename
FROM EMP
WHERE salary > 20000 ;
```

SQL Main Components - DCL

DCL - Data Control Language

(used for creating table permissions, valid users, etc)

e.g. *(assuming John is a registered user)*

```
GRANT ALL [PRIVILEGES ]
ON EMP
TO John
[WITH GRANT OPTION];
```

SQL Reserved Words Table & Operators

- **1. SQL Reserved Words Table -** This section lists **common SQL keywords**, which are essential for writing SQL queries. These include:
  - **Commands:** SELECT, UPDATE, DELETE, INSERT, HAVING, SET
  - **Clauses:** WHERE, FROM, GROUP BY, ORDER BY, DISTINCT
  - **Logical Operators:** AND, OR, NOT
  - **Sorting:** ASC (Ascending), DESC (Descending)
  - **Other:** VALUES, INTO, AS, IN

2) **Comparison Operators -** This section describes **operators used in SQL for filtering and comparisons**:
   - = → **Equal to**
   - <> → **Not equal to** (alternative: !=)
   - > → **Greater than**
   - < → **Less than**
   - >= → **Greater than or equal to**
   - <= → **Less than or equal to**

Common Data Types

Some Common Data Types

Character Strings:

char(n) - data entries are consistent in size e.g. char (12)

varchar(n) - data entries vary considerably in size e.g. varchar (30)

Exact Numerics:

tinyint (0 to 255)

smallint (-32,768 to 32,767),

int (-2,147,483,648 to 2,147,483,647)

bigint (-2^63 to 2^63-1)

decimal [(p[,s])] – precision and scale e.g. decimal (7,2)

Dates and Times e.g. date = '1912-10-25';

(See the MySQL Language Reference for the Complete List)

CREATE TABLE & primary keys

Building a Database – Creating the Tables

- **CREATE TABLE** EMP **(**empnum Smallint **Not Null**, ename CHAR(15), salary SMALLMONEY, deptnum SMALLINT**);**

- This SQL statement creates a table named **EMP** (which likely stands for "Employee") with four columns (Attributes).

```
CREATE TABLE EMP (
    empnum SMALLINT NOT NULL,
    ename CHAR(15),
    salary SMALLMONEY,
    deptnum SMALLINT
);
```

Building a Database – Creating the Tables

- **CREATE TABLE EMP (empnum SMALLINT NOT NULL, ename CHAR(15), salary SMALLMONEY, deptnum SMALLINT);**

✓ **empnum (SMALLINT NOT NULL)**
  - SMALLINT: A small integer data type (typically 2 bytes, allowing values from -32,768 to 32,767).
  - NOT NULL: Ensures that this column **must** have a value and cannot be left empty.

✓ **ename (CHAR(15))**
  - CHAR(15): A fixed-length character data type that stores exactly **15** characters (padded with spaces if the name is shorter).
  - Used for storing employee names.

✓ **salary (SMALLMONEY)**
  - SMALLMONEY: A data type specific to **MS SQL Server**, used for monetary values
    {DECIMAL(10,2) used in MySQL}
  - It can store values ranging from **-214,748.3648 to 214,748.3647**, making it useful for storing salaries with decimal precision.

✓ **deptnum (SMALLINT)**
  - SMALLINT: Again, a small integer type, used to store the department number the employee belongs to.

Activity

- What about the Keys
  - Which one is the PRIMAR KEY
  - Which one is the FOREIGN KEY

```
CREATE TABLE EMP (
    empnum SMALLINT NOT NULL,
    ename CHAR(15),
    salary SMALLMONEY,
    deptnum SMALLINT
);
```

Building a Database – Creating the Tables

Consider the following relational schema:

EMPLOYEE (<u>enum</u> , ename, salary, floor)
PROJECT (<u>pnum</u> , pname, leader)
WORKS_ON (<u>enum</u>* , <u>pnum</u>* , role)

With sample data as shown

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

PROJECT

pnum	pname	leader
121	IT	Gates
135	Design	Sinclair
147	Analysis	Einstein
216	Publicity	Saatchi
227	Theatre	Dench
251	Sport	Shearer

Notice the PKs and FKs, and how the tables will be linked

Decide on the Data Types

Can an attribute be **NULL**?

Now, write the SQL to create each table

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager

1. Create the EMPLOYEE Table

EMPLOYEE (<u>enum</u> , ename, salary, floor)

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

SQL Style Guide:

Table names and SQL Reserved words – ALL CAPS, words separated by underscore (_)

Attribute names – meaningful, all lowercase, words separated by underscore (_)

Layout – generally, one SQL clause per line

Don't waste storage space – *choose data types that are just large enough.*

```
CREATE TABLE EMPLOYEE (
    enum  int  not null,
    ename char (15),
    salary decimal (7,2),
    floor tinyint,
    CONSTRAINT pk_emp PRIMARY KEY (enum)
);
```

Entity Integrity - no *attributes participating in the primary key of a relation can accept null values.*

Primary Key Constraint – *can be added when the table is created, or added later using an ALTER TABLE statement.*

In this example, pk_emp is the **name** given to the Primary Key Constraint

2. Create the WORK_ON Table

WORKS_ON (enum* , pnum* , role)

```
CREATE TABLE WORKS_ON (
   enum  int  not null,
   pnum smallint not null,
   role char(15),
   CONSTRAINT  pk_wo PRIMARY KEY (enum,
   pnum)
);
```

We will use an ALTER TABLE statement to make enum and pnum **FKs**

Data Types must be the same as PKs in corresponding tables

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager

3. Create the PROJECT Table

3. Create the PROJECT Table

PROJECT (pnum , pname, leader)

PROJECT

pnum	pname	leader
121	IT	Gates
135	Design	Sinclair
147	Analysis	Einstein
216	Publicity	Saatchi
227	Theatre	Dench
251	Sport	Shearer

```
CREATE TABLE PROJECT (
    pnum  smallint  not null,
    pname  char(15),
    leader  char(15),
    CONSTRAINT  pk_proj PRIMARY KEY (pnum)
);
```

Integrity constraints (DEFAULT, UNIQUE, CHECK)

Integrity Constraints

- What are Integrity Constraints?
  - Constraints enforce business rules on tables.
  - Can be applied using **CREATE TABLE or later via ALTER TABLE**.

- Common SQL Constraints:
  - **NOT NULL** → Ensures that a column cannot have NULL values.
  - **PRIMARY KEY** → Ensures that each row is uniquely identifiable.

- Constraints can be named – as earlier with **PRIMARY KEY** in the **CREATE TABLE** statements – or applied directly to a column/attribute – as with **not null**.

Integrity Constraints

Other Important Constraints:

- **DEFAULT** → Assigns a default value if no value is provided.
  - Example: role CHAR(15) **DEFAULT** 'To be set'
- **UNIQUE** → Ensures all values in a column are **distinct**.
  - Example: leader CHAR(15) **UNIQUE**
- **CHECK** → Ensures that values meet a **specific condition**.
  - Example: floor TINYINT **CHECK** (floor <= 7)

Foreign keys & ALTER TABLE

Why Use ALTER TABLE?

- The **ALTER TABLE statement** is used to modify the structure of an existing table without deleting or recreating it. It allows adding, modifying, deleting columns, renaming tables, and adding constraints.
  - Allows modifying a table without losing data.
  - Useful when updating database structures in a live system.
  - Helps maintain data integrity with constraints.
- Have three Formulas:
  - For adding: ALTER TABLE …… ADD ……;
  - For modifying: ALTER TABLE …… MODIFY / CHANGE / RENAMER……;
  - For deleting: ALTER TABLE …… DROP ……;

ALTER TABLE (Adding the Foreign Key)

Building a Database – Create the Links (FKs) and establish Referential Integrity

Relational Schema: **EMPLOYEE** (enum , ename, salary, floor)
PROJECT (pnum , pname, leader)
WORKS_ON (enum*, pnum*, role)

Remember: If a relational table includes a foreign key (**FK**) matching the primary key (**PK**) of another relational table, then every value of the FK **must**:

- either be equal to a value of the PK in some tuple (row)
- or be wholly **NULL**

Summary of ALTER TABLE Operations

Operation	Usage	Example
`ADD`	Add a new column	`ADD department VARCHAR(20);`
	Add a constraint	`ADD CONSTRAINT fk_dept FOREIGN KEY (department) REFERENCES DEPARTMENT(dept_id);`
`ALTER`	Change column type	`MODIFY salary DECIMAL(10,2);`
	Rename column	`CHANGE COLUMN ename full_name VARCHAR(30);`
	Rename table	`RENAME TO EMP;`
`DROP`	Delete a column	`DROP COLUMN floor;`
	Delete a constraint	`DROP FOREIGN KEY fk_dept;`

In MySql, ALTER syntax is not valid, but it is valid in MS SQL

ALTER TABLE (Adding the Foreign Key)

- Foreign Keys can be created, and **Referential Integrity** established, using an **ALTER TABLE** statement
- Requires referencing **from** the **Foreign Key towards** a corresponding **Primary Key** in another table

- To put referential integrity between **enum** from **WORKS_ON** (**FK**) and **enum** from **EMPLOYEE** (**PK**)

```
ALTER TABLE WORKS_ON
ADD CONSTRAINT fk_wo1 FOREIGN KEY (enum) REFERENCES EMPLOYEE (enum);
```

Attributes don't have to have the same name,
but **MUST** have the same Data Type

- To put referential integrity between **pnum** from **WORKS_ON** (**FK**) & **pnum** from **PROJECT** (**PK**)

```
ALTER TABLE WORKS_ON
ADD CONSTRAINT fk_wo2 FOREIGN KEY (pnum) REFERENCES PROJECT (pnum);
```

ALTER TABLE

Other useful ALTER TABLE Commands

Changing a datatype/size (to whole numbers only)

In general, may not be possible. Depends on data types and if data has been entered already.

```
ALTER TABLE EMPLOYEE
CHANGE COLUMN salary int;
```

Adding a new attribute (budget to **PROJECT**)

```
ALTER TABLE PROJECT
ADD budget decimal (9,2);
```

Delete budget attribute from **PROJECT**

```
ALTER TABLE PROJECT
DROP COLUMN budget;
```

Adding a PK (if not done in create table)

```
ALTER TABLE EMPLOYEE
ADD CONSTRIANT pk_emp PRIMARY KEY (enum);
```

Drop the PK from **PROJECT** (A two stage process:

1. Drop referential Integrity on PROJECT PK – refer by **name**
2. Drop the PK from PROJECT – refer by **name**)

```
ALTER TABLE WORKS_ON          (drop referential integrity)
DROP CONSTRAINT fk_wo2;

ALTER TABLE PROJECT           (drop PK)
DROP CONSTRAINT pk_proj;
```

INSERT, UPDATE & DELETE

Building a Database – Adding Records

General Syntax

INSERT INTO TABLENAME (attribute list)
VALUES (record values)

e.g. Add new employee record 852559 , 'Brown', 18000, 4

INSERT INTO EMPLOYEE (enum, ename, salary, floor)
VALUES (852559, 'Brown', 18000, 4) ;

Which can be shortened where a whole record is added to:

INSERT INTO EMPLOYEE
VALUES (852559, 'Brown', 18000, 4);

Building a Database – Deleting Records

Building a Database – Deleting Records

General Syntax

DELETE FROM TABLENAME
WHERE selection clause;

e.g. Delete employees earning above 18000 on the 2nd floor

DELETE FROM EMPLOYEE
WHERE salary > 18000
AND floor = 2 ;

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

Building a Database – Updating Records

General Syntax

```
UPDATE TABLENAME
SET attribute = new value
WHERE selection clause;
```

e.g. Change the project leader of the Theatre project to Branagh

```
UPDATE PROJECT
SET leader = 'Branagh'
WHERE pname = ‘theatre';
```

Building a Database – Editing Records

NOTE – **Referential Integrity** must be considered when **adding**, **deleting** or **updating** records

Using the relational schema and sample data on slide 4, consider each of the examples above. Will the queries execute correctly?

1. *Add new employee record 852559 , 'Brown', 18000, 4*
2. *Delete employees earning above 18000 on the 2nd floor*
3. *Change the project leader of the Theatre project to Branagh*

```
UPDATE PROJECT
SET leader = 'Branagh‘
WHERE pname = ‘theatre';
```

Building a Database – Editing Records

Consider the following relational schema:

EMPLOYEE (enum , ename, salary, floor)
PROJECT (pnum , pname, leader)
WORKS_ON (enum* , pnum* , role)

With sample data as shown

Will the following queries execute correctly?

```
INSERT INTO EMPLOYEE
VALUES (852559, 'Brown', 18000, 4);
```

```
DELETE FROM EMPLOYEE
WHERE salary > 18000
AND floor = 2 ;
```

```
UPDATE PROJECT
SET leader = 'Branagh'
WHERE pname = 'Theatre';
```

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

PROJECT

pnum	pname	leader
121	IT	Gates
135	Design	Sinclair
147	Analysis	Einstein
216	Publicity	Saatchi
227	Theatre	Dench
251	Sport	Shearer

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager



Summary

- **SQL components:** DDL defines database structures, DML retrieves and changes data, and DCL manages permissions.
- **Creating tables:** Use `CREATE TABLE` with suitable data types and primary keys, including composite keys.
- **Integrity constraints:** Apply `NOT NULL`, `DEFAULT`, `UNIQUE` and `CHECK` to enforce data rules.
- **Linking tables:** Foreign keys establish relationships and maintain referential integrity.
- **Changing structures:** Use `ALTER TABLE` to add, modify or remove columns and constraints.
- **Managing records:** Use `INSERT`, `UPDATE` and `DELETE`, with `WHERE` to target specific records.




THE
END